

Assignment 2:
Name: Arik Gershman

This notebook is to help finish Lucas-Kanade algorithm

```
In [1]: import cv2
import numpy as np
from scipy import ndimage
import matplotlib.pyplot as plt
from scipy.interpolate import RectBivariateSpline
import os, sys, time
%matplotlib inline
```

Feature Tracker (100 points)

Overview

In the problem, you will implement a corner detector and feature tracker that track features from the image sequence hotel. Since this is a two part problem, we have included precomputed intermediate results in the *Data* section in case you're unable to complete any portion.

Note: Do not use existing keypoint detectors, trackers, or structure from motion code, such as found on the web, and OpenCV.

```
In [2]: # Download Data -- run this cell only one time per runtime
import gdown
url = "https://drive.google.com/uc?id=1L5oV8H7EEh5ggnc0YThBB40XdGYPozqA"
output = "HW2_files.zip" # Change this to your desired file name
gdown.download(url, output, quiet=False)
!unzip "/content/HW2_files.zip" -d "/content/"

# # Load Initial Key Points
data = np.load('/content/HW2_files/initial_keypoints.npz')
X0 = data['X0']
Y0 = data['Y0']

print(X0.shape)
print(Y0.shape)
key_pts = np.concatenate((X0, Y0), axis=1)
print(key_pts.shape)
print(key_pts)

Downloading...
From: https://drive.google.com/uc?id=1L5oV8H7EEh5ggnc0YThBB40XdGYPozqA
To: /content/HW2_files.zip
100%[██████████] 5.12M/5.12M [00:00<00:00, 87.2MB/s]
```

```

Archive: /content/HW2_files.zip
  creating: /content/HW2_files/
  inflating: /content/HW2_files/_DS_Store
  inflating: /content/_MACOSX/HW2_files/_._DS_Store
  creating: /content/HW2_files/images/
  inflating: /content/_MACOSX/HW2_files/_._images
  inflating: /content/HW2_files/initial_keypoints.npz
  inflating: /content/_MACOSX/HW2_files/_._initial_keypoints.npz
  inflating: /content/HW2_files/images/hotel.seq41.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq41.png
  inflating: /content/HW2_files/images/hotel.seq40.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq40.png
  inflating: /content/HW2_files/images/hotel.seq42.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq42.png
  inflating: /content/HW2_files/images/hotel.seq43.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq43.png
  inflating: /content/HW2_files/images/hotel.seq47.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq47.png
  inflating: /content/HW2_files/images/hotel.seq46.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq46.png
  inflating: /content/HW2_files/images/hotel.seq50.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq50.png
  inflating: /content/HW2_files/images/hotel.seq44.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq44.png
  inflating: /content/HW2_files/images/hotel.seq8.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq8.png
  inflating: /content/HW2_files/images/hotel.seq9.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq9.png
  inflating: /content/HW2_files/images/hotel.seq45.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq45.png
  inflating: /content/HW2_files/images/hotel.seq36.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq36.png
  inflating: /content/HW2_files/images/hotel.seq22.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq22.png
  inflating: /content/HW2_files/images/hotel.seq23.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq23.png
  inflating: /content/HW2_files/images/hotel.seq37.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq37.png
  inflating: /content/HW2_files/images/hotel.seq21.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq21.png
  inflating: /content/HW2_files/images/hotel.seq35.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq35.png
  inflating: /content/HW2_files/images/hotel.seq34.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq34.png
  inflating: /content/HW2_files/images/hotel.seq20.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq20.png
  inflating: /content/HW2_files/images/hotel.seq24.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq24.png
  inflating: /content/HW2_files/images/hotel.seq30.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq30.png
  inflating: /content/HW2_files/images/hotel.seq18.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq18.png
  inflating: /content/HW2_files/images/hotel.seq19.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq19.png
  inflating: /content/HW2_files/images/hotel.seq31.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq31.png
  inflating: /content/HW2_files/images/hotel.seq25.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq25.png
  inflating: /content/HW2_files/images/hotel.seq33.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq33.png
  inflating: /content/HW2_files/images/hotel.seq27.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq27.png
  inflating: /content/HW2_files/images/hotel.seq26.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq26.png
  inflating: /content/HW2_files/images/hotel.seq32.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq32.png
  inflating: /content/HW2_files/images/hotel.seq17.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq17.png
  inflating: /content/HW2_files/images/hotel.seq16.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq16.png
  inflating: /content/HW2_files/images/hotel.seq28.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq28.png
  inflating: /content/HW2_files/images/hotel.seq14.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq14.png
  inflating: /content/HW2_files/images/hotel.seq15.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq15.png
  inflating: /content/HW2_files/images/hotel.seq29.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq29.png
  inflating: /content/HW2_files/images/hotel.seq11.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq11.png
  inflating: /content/HW2_files/images/hotel.seq39.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq39.png
  inflating: /content/HW2_files/images/hotel.seq38.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq38.png
  inflating: /content/HW2_files/images/hotel.seq10.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq10.png
  inflating: /content/HW2_files/images/hotel.seq12.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq12.png
  inflating: /content/HW2_files/images/hotel.seq13.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq13.png
  inflating: /content/HW2_files/images/hotel.seq48.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq48.png
  inflating: /content/HW2_files/images/hotel.seq4.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq4.png
  inflating: /content/HW2_files/images/hotel.seq5.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq5.png
  inflating: /content/HW2_files/images/hotel.seq49.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq49.png
  inflating: /content/HW2_files/images/hotel.seq7.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq7.png
  inflating: /content/HW2_files/images/hotel.seq6.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq6.png
  inflating: /content/HW2_files/images/hotel.seq3.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq3.png
  inflating: /content/HW2_files/images/hotel.seq1.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq1.png
  inflating: /content/HW2_files/images/hotel.seq0.png
  inflating: /content/_MACOSX/HW2_files/images/_._hotel.seq0.png
(555, 1)
(555, 1)
(555, 2)
[[ 4 471]
 [ 9 479]
 [13 315]
 ...
[506 448]
[510 242]
[512 420]]

```

```

In [28]: def readImages(folder, num_images):
         arr_images = []
         for i in range(num_images):
             arr_images.append(cv2.imread(f'{folder}hotel.seq{i}.png'))
         return np.array(arr_images, dtype=np.float32)

# read all 51 sequences of images
folder = '/content/HW2_files/images/'
im = readImages(folder, 51)

```

1.1 Keypoint Selection - Harris Corner Detection (30 pts)

In the class, we know that we approximate the Sum Square Distance (SSD) error by Tylor expansion in to this form:

where $w_{x,y}$ is 2D Gaussian filter, and the H matrix:

In this section, you will need to implement the function of Harris Corner Detection by constructing the approximated SSD error matrix and Harris operator:

$$f = \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2} \quad (1)$$

$$= \frac{\text{determinant}(H)}{\text{trace}(H)}$$

Follow the instructions in the code block, check with the formula above if needed.

Hint:

- Choose a proper threshold so that edges and noisy patches are ignored.
- Do local non-maxima suppression over a 5x5 window centered at each point.

This should give several hundred good points to track.

Code - Harris Corner Detection (10 pts)

```
In [29]: def HarrisCornerDet(img, tau):
    ...
    Detecting keypoints using Harris corner criterion

    Parameters:
        img: Input image
        tau: threshold for Harris operator f

    Steps:
    1. Convert image to grayscale
    2. Smooth the image
    3. Compute image gradients: Ix, Iy, IxIy
    4. Create structure tensor H = [ I(Ix^2)  I(IxIy) ]
        [ I(IxIy)  I(Iy^2) ]
    5. Compute Harris operator f = determinant(H) / trace(H)
        - Hint: Remember to normalize f to [0,1]
    6. Non-maximum suppression
        - Hint: You can use ndimage.maximum_filter to create max_filter

    * Remember to add plots below for the write-up

    Output:
        keypoints : (N,2) array of [x,y] keypoints
    ...

    # Parameters given
    smooth_window_size = 7
    nms_window_size = 5

    ##### YOUR CODE HERE #####
    # Step 1: Convert image to grayscale
    img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).astype(np.float32)

    # Step 2: Smooth the image
    img = cv2.GaussianBlur(img, (smooth_window_size, smooth_window_size), 0)

    # Step 3: Compute image gradients: Ix^2, Iy^2, IxIy
    img_shifted_right = np.roll(img, 1, axis=1)
    img_shifted_right[:, 0] = 0

    img_shifted_left = np.roll(img, -1, axis=1)
    img_shifted_left[:, -1] = 0

    img_shifted_up = np.roll(img, -1, axis=0)
    img_shifted_up[-1, :] = 0

    img_shifted_down = np.roll(img, 1, axis=0)
    img_shifted_down[0, :] = 0

    Ix = (img_shifted_left - img_shifted_right) / 2
    Iy = (img_shifted_up - img_shifted_down) / 2

    IxIy = Ix*Iy
    Ix2 = Ix**2
    Iy2 = Iy**2

    # Step 4: Prepare elements for Structure Tensor H
    # H = [ I(Ix^2)  I(IxIy) ]
    #       [ I(IxIy)  I(Iy^2) ]
    GIx2 = cv2.GaussianBlur(Ix2, (smooth_window_size, smooth_window_size), 0)
    GIy2 = cv2.GaussianBlur(Iy2, (smooth_window_size, smooth_window_size), 0)
    GIXIy = cv2.GaussianBlur(IxIy, (smooth_window_size, smooth_window_size), 0)

    # Step 5: Compute Harris operator f = determinant(H) / trace(H)
    # - Hint: Remember to normalize f to [0,1]
    det_H = (GIx2 * GIy2) - (GIXIy * GIXIy)
    trace_H = GIx2 + GIy2
    f = det_H / trace_H
    f = (f - f.min()) / (f.max() - f.min())

    # Step 6: Non-maximum suppression
    # - Hint: You can use ndimage.maximum_filter to create max_filter
    max_filter = ndimage.maximum_filter(f, nms_window_size)
    y, x = np.where((f == max_filter) & (f > tau))

    key_pts = np.column_stack((x,y))

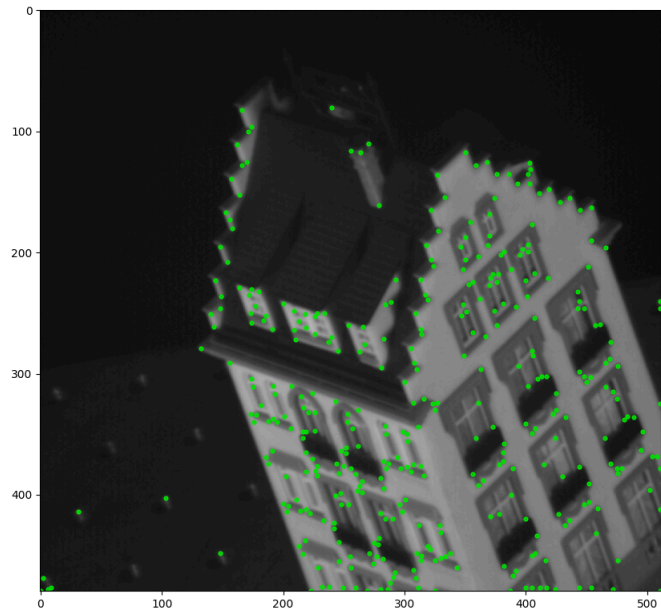
    return key_pts
```

Test your codes

```
In [30]: # Parameter setting
tau = 0.06

# Run Harris corner detection
key_pts_detected = HarrisCornerDet(im[0], tau)

# add plots for the write-up
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(im[0] / 255)
plt.scatter(key_pts_detected[:, 0], key_pts_detected[:, 1], color=(0,0.9,0), linewidths=3, s=2)
plt.show()
```



Reference result

- Your result should look similar to this figure:

Write-up (40 pts)

- (5 pts) Explain your implementation of HarrisCornerDet()
- (5 pts) Display the first frame of the sequence overlaid with the detected keypoints. Ensure that they are clearly visible (plot with `color='g'` and `linewidths=3`)

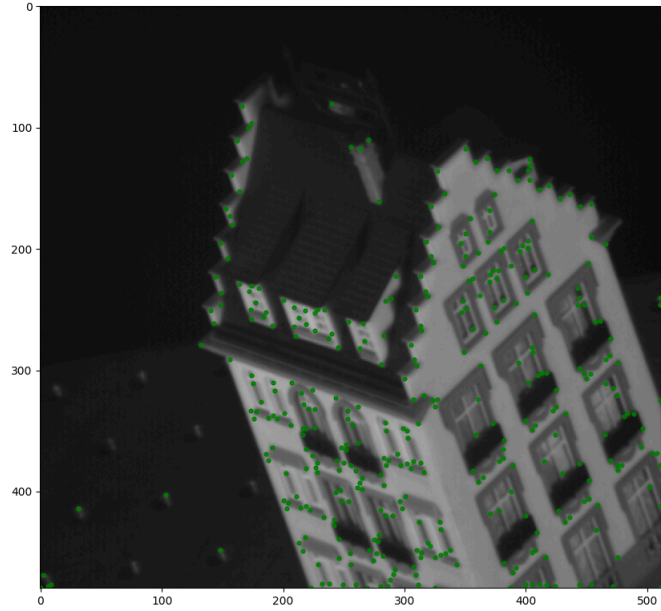
Include your write-up here

1. First, I convert the image to grayscale and smooth/blur it to remove some of the high-frequency details that are unnecessary for the corner detection. Then, I use cv2.Sobel to compute the gradients of each pixel and construct the ix^2 , ly^2 , and $ixly$ matrices. Instead of summing over a local window W , I use the trick from the slides and apply a Gaussian blur instead to aggregate the gradient information. Then, instead of calculating the exact eigenvalues of these matrices, I estimate the minimum eigenvalues using the Harris trick. Also, instead of directly constructing the H tensor and compute the determinant and trace of each matrix, I compute the determinant and trace of each 2×2 matrix in H using the mathematical formulas for determinants and traces of 2×2 matrices, taking advantage of NumPy: $\det = a \cdot d - b \cdot c$ and $\text{trace} = a + d$. I then compute the Harris operator f as $\det/\text{trace}$ and normalize it to $[0,1]$. Then, I create the `max_filter` using `ndimage.maximum_filter`, and extract the coordinates that are both a maximum point in their window and have a value greater than the parameter `tau`. This leaves just the corners that are worth tracking without much noise around them. I then return the coordinates as a $N \times 2$ matrix where each row corresponds to a corner's coordinate.

```
In [31]: # 2.
# Parameter setting
tau = 0.06

# Run Harris corner detection
key_pts_detected = HarrisCornerDet(im[0], tau)

# add plots for the write-up
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(im[0] / 255)
plt.scatter(key_pts_detected[:, 0], key_pts_detected[:, 1], color='g', linewidths=3, s=2)
plt.show()
```



1.2 Feature Tracking (70 pts)

Description

- Apply the Kanade-Lucas-Tomasi tracking procedure to track the keypoints found in part 1.1 (or the given keypoints in the *Data* section) throughout the hotel sequence.
- Steps for Iterative Lucas-Kanade

- Some keypoints will move out of the image frame over the course of the sequence. Discard keypoints for which the predicted translation falls outside the image frame.

Hint:

- How to get (u, v)? Solve $Ax=b$. You can utilize `np.linalg.solve()`

- From the 1st frame to the 2nd frame, use the detected keypoints at the first frame as initialization points. From the 2nd to 3rd frame, use the tracked positions at the 2nd frame as initialization. Note that the tracked keypoints in general are not at integer positions.
- For each point, use a window size of 15 x 15.

Add code to **plot** your tracked points overlayed in the **first sequence** and the **last sequence**. They should look similar to the second picture shown in the Overview section.

Code - Lucas-Kanade

Helper Function

```
In [32]: """
Feel free to use the functions below in your implementation.
It's not necessarily to use them, you can have your own implementation
"""

def resize(im, scale):
    if scale < 1:
        im = cv2.GaussianBlur(im, (9, 9), 2)
    im = cv2.resize(im, None, fx=scale, fy=scale, interpolation=cv2.INTER_LINEAR)
    return im

def get_windows(spline, cx, cy, half_size):
    x_grids = np.linspace(cx - half_size, cx + half_size, half_size * 2 + 1).transpose()
    y_grids = np.linspace(cy - half_size, cy + half_size, half_size * 2 + 1).transpose()
    samples = []
    for x_grid, y_grid in zip(x_grids, y_grids):
        x_meshgrid, y_meshgrid = np.meshgrid(x_grid, y_grid, indexing='xy')
        sample = spline.ev(xi=x_meshgrid, yi=y_meshgrid)
        samples.append(sample)
    return np.stack(samples)
```

Complete the code - Lucas-Kanade

```
In [33]: def compute_gradients(img):
    """
    Computes the image gradients in the x and y directions using the central difference method.

    Parameters:
    -----
    img : numpy.ndarray
        Input grayscale image of shape (H, W), where H is the height and W is the width.

    Returns:
    -----
    Ix : numpy.ndarray
        Gradient of the image in the x-direction, shape (H, W).

    Iy : numpy.ndarray
        Gradient of the image in the y-direction, shape (H, W).

    Notes:
    -----
    - The function applies the central difference formula:
        Ix(x, y) = (I(x+1, y) - I(x-1, y)) / 2
        Iy(x, y) = (I(x, y+1) - I(x, y-1)) / 2
    - You can do zero-padding for the boundary pixels if needed.
    """
    ### YOUR CODE HERE ###
    img = img.astype(np.float32)

    img_shifted_right = np.roll(img, 1, axis=1)
    img_shifted_right[:, 0] = 0

    img_shifted_left = np.roll(img, -1, axis=1)
    img_shifted_left[:, -1] = 0

    img_shifted_up = np.roll(img, -1, axis=0)
    img_shifted_up[-1, :] = 0

    img_shifted_down = np.roll(img, 1, axis=0)
    img_shifted_down[0, :] = 0

    Ix = (img_shifted_left - img_shifted_right) / 2
    Iy = (img_shifted_up - img_shifted_down) / 2

    ### YOUR CODE HERE ###
    return Ix, Iy

def valid_range_check(x, y, height, width, ws):
    """
    Checks whether the given points (x, y) are within the valid range of the image.

    Parameters:
    -----
    x : numpy.ndarray or int
        X-coordinates of the points to check, shape (N,) or (1,).

    y : numpy.ndarray or int
        Y-coordinates of the points to check, shape (N,) or (1,).

    height : int
        Height of the image.

    width : int
        Width of the image.

    ws : int
        Window size, defining the margin around the image boundary where points are considered invalid.

    Returns:
    -----
    valid_mask : numpy.ndarray
        Boolean mask indicating whether each point is within the valid range, shape (N,).

    Notes:
    -----
    - A point (x, y) is considered valid if:
        ws ≤ x < width - ws
        ws ≤ y < height - ws
    - If 'x' and 'y' are numpy arrays, the function returns an array of the same shape.
    """
    ### YOUR CODE HERE ###
    valid_mask = (ws <= x) & (x < width - ws) & (ws <= y) & (y < height - ws)

    return valid_mask
    ### YOUR CODE HERE ###

def iterative_Lucaskanade(x1, y1, img1, img2, ws, iteration=10, thresh=0.4):
    """
    Performs iterative Lucas-Kanade feature tracking to estimate optical flow.
```

```

Parameters:
-----
x1 : numpy.ndarray
    Initial x-coordinates of feature points, shape (N,).

y1 : numpy.ndarray
    Initial y-coordinates of feature points, shape (N,).

img1 : numpy.ndarray
    First (reference) grayscale image, shape (H, W).

img2 : numpy.ndarray
    Second grayscale image (target frame for tracking), shape (H, W).

ws : int
    Window size around each feature point for computing local gradients.

iteration : int, optional (default=10)
    Maximum number of iterations for iterative refinement.

thresh : float, optional (default=0.4)
    Convergence threshold; tracking stops if the mean motion change is below this value.

Returns:
-----
x2 : numpy.ndarray
    Estimated x-coordinates of tracked feature points in img2, shape (N,).

y2 : numpy.ndarray
    Estimated y-coordinates of tracked feature points in img2, shape (N,).

Notes:
-----
- The method initializes points at (x1, y1) and refines them iteratively using local gradients.
- Image gradients are computed using central differences.
- A least-squares system is solved for each feature point to estimate movement.
- The tracking is refined until either 'iteration' is reached or displacement change is below 'thresh'.
- Uses 'RectBivariateSpline' for subpixel interpolation.
"""
# Step 0: Initialize points. No need to change.
x2 = x1.copy()
y2 = y1.copy()

N = x1.shape[0]
H, W = img1.shape

##### YOUR CODE HERE #####
# Step 1: Calculate gradients Ix2, Iy2, Ixy. TODO: Finish 'compute_gradients()'
Ix, Iy = compute_gradients(img1)

# TODO: Compute Ix2 = Ix * Ix, Iy2 = Iy * Iy, Ixy = Ix * Iy
Ix2 = Ix*Ix
Iy2 = Iy*Iy
Ixy = Ix*Iy

# Step 2: Setup interpolation for gradient and intensity. No need to change.
y_grid = np.arange(0, H)
x_grid = np.arange(0, W)
# RectBivariateSpline() provides an interpolation sampler so that you can sample sub-pixel values on a grid
spline_Ix = RectBivariateSpline(x=x_grid, y=y_grid, z=Ix.transpose(1, 0))
spline_Iy = RectBivariateSpline(x=x_grid, y=y_grid, z=Iy.transpose(1, 0))
spline_img1 = RectBivariateSpline(x=x_grid, y=y_grid, z=img1.transpose(1, 0))
spline_img2 = RectBivariateSpline(x=x_grid, y=y_grid, z=img2.transpose(1, 0))

# Step 3: Iterate through iterations
for i in range(iteration):
    # Step 3.1: Get points within valid range. TODO: Finish valid_range_check()
    mask1 = valid_range_check(x1, y1, H, W, ws)
    mask2 = valid_range_check(x2, y2, H, W, ws)
    mask = mask1 & mask2
    x1_ = x1[mask]
    y1_ = y1[mask]
    x2_ = x2[mask]
    y2_ = y2[mask]
    assert mask.sum() > 0

    uv = np.zeros((len(x1_), 2)) # Store displacement per point
    # Traverse each point
    for idx in range(len(x1_)):
        # Step 3.2: Calculate matrix ATA and ATb.
        x, y = x1_[idx:idx+1], y1_[idx:idx+1]
        x_next, y_next = x2_[idx:idx+1], y2_[idx:idx+1]

        # Get windowed gradient values
        Ix_window = get_windows(spline_Ix, x, y, ws)
        Iy_window = get_windows(spline_Iy, x, y, ws)

        # Compute elements of the structure tensor
        # TODO: compute sum for Ix2, Ixy, Iy2 for each window
        # sum over (Ix_windows * Ix_windows)
        # sum over (Iy_windows * Iy_windows)
        # sum over (Ix_windows * Iy_windows)
        Ixx_sum = np.sum(Ix_window * Ix_window)
        Iyy_sum = np.sum(Iy_window * Iy_window)
        Ixy_sum = np.sum(Ix_window * Iy_window)

        # Compute intensity difference
        t1_window = get_windows(spline_img1, x, y, ws)
        t2_window = get_windows(spline_img2, x_next, y_next, ws)

        # TODO: update the temporal gradient It
        It_window = t2_window - t1_window

        # TODO: compute sum over
        # Ix_window * It_window
        # Iy_window * It_window
        Ixt_sum = np.sum(Ix_window * It_window)
        Iyt_sum = np.sum(Iy_window * It_window)

        # Step 3.3: Solve least squares by getting ATAx = ATb: x = (ATA)^(-1)ATb
        # - Hint: solve by np.linalg.solve()
        # ATA = # shape: (2, 2)
        # ATb = # shape (2,)
        # uv[idx] = np.linalg.solve(ATA, ATb)
        ATA = [[Ixx_sum, Ixy_sum],
               [Ixy_sum, Iyy_sum]]
        ATb = [-Ixt_sum, -Iyt_sum]
        uv[idx] = np.linalg.solve(ATA, ATb)

        # Step 3.4 Update u, v movement to x2, y2 to get x2_, y2_
        x2_[idx] = x_next + uv[idx][0] #complete here
        y2_[idx] = y_next + uv[idx][1] #complete here

    # early stop if the shift changes little. no need to change below
    shift = np.sqrt(np.square(uv[:, 0]) + np.square(uv[:, 1])).mean()
    print('iteration:', i, ' ', shift_uv_mean:', shift)

    if shift < thresh:
        break

    # assign values to valid points.
    x2[mask] = x2_
    y2[mask] = y2_

return x2, y2

#no need to change this code
def getNextPoints(x1, y1, img1, img2, ws, iteration=10):

```

```

'''
Iterative Lucas-Kanade feature tracking
x, y : initialized keypoint position in im2
ws: patch window size
iteration: parameter to control how many iterations to run in iterative_LucasKanade function

output: tracked keypoint positions in im2
'''

# Step 1: convert RGB images to gray image and apply Gaussian blur
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
img1 = cv2.GaussianBlur(img1, (13, 13), 3)
img2 = cv2.GaussianBlur(img2, (13, 13), 3)

# Step 2: Iterative Lucas-Kanade feature tracking
x2, y2 = iterative_LucasKanade(x1, y1, img1, img2, ws, iteration)

return x2, y2

```

Code below for checking the iterative_Lucaskanade, no need to change any code below

Sanity Check 1: compute_gradients()

You gradients should look similar to cv2.Sobel(). There is a slight difference between two because of cv2's own implementation.

```

In [34]: # Compute gradients using NumPy central difference
Ix_central, Iy_central = compute_gradients(im[0].astype(np.float64))

# Compute gradients using OpenCV Sobel (ksize=3 for central difference)
Ix_sobel = cv2.Sobel(im[0].astype(np.float64), cv2.CV_64F, 1, 0, ksize=3) / 8.0 # Normalize by 8 to match central diff scaling
Iy_sobel = cv2.Sobel(im[0].astype(np.float64), cv2.CV_64F, 0, 1, ksize=3) / 8.0

# Plot comparison
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].imshow(Ix_central, cmap='gray')
axes[0, 0].set_title('Ix (Central Difference)')

axes[0, 1].imshow(Ix_sobel, cmap='gray')
axes[0, 1].set_title('Ix (cv2.Sobel)')

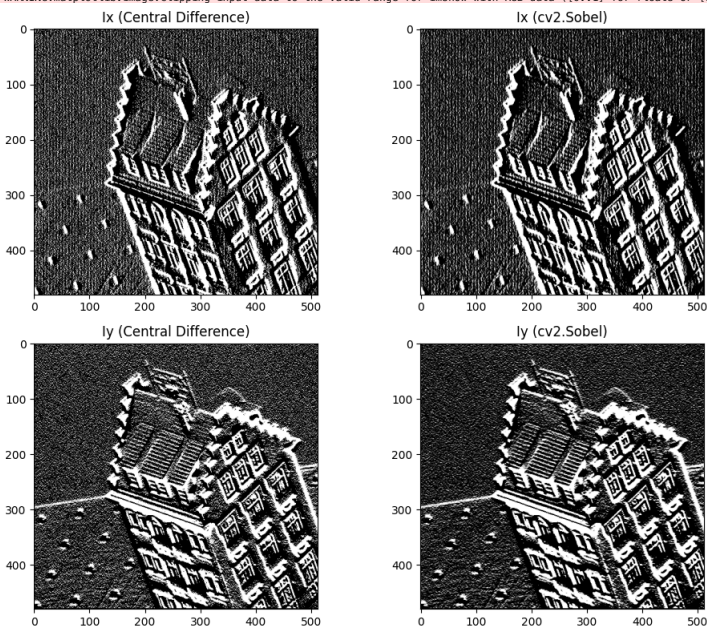
axes[1, 0].imshow(Iy_central, cmap='gray')
axes[1, 0].set_title('Iy (Central Difference)')

axes[1, 1].imshow(Iy_sobel, cmap='gray')
axes[1, 1].set_title('Iy (cv2.Sobel)')

plt.tight_layout()
plt.show()

```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-63.0..36.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-34.25..35.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-75.0..39.0].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-31.25..37.375].



Sanity Check 2: valid_range_check()

```

In [35]: import unittest

class TestValidRangeCheck(unittest.TestCase):
    def test_inside_valid_range(self):
        self.assertTrue(valid_range_check(5, 5, 10, 10, 2), f"Expected True for (5, 5) in 10x10 with ws=2")
        self.assertTrue(valid_range_check(3, 3, 8, 8, 2), f"Expected True for (3, 3) in 8x8 with ws=2")

    def test_outside_valid_range(self):
        self.assertFalse(valid_range_check(1, 1, 10, 10, 2), f"Expected False for (1, 1) in 10x10 with ws=2")
        self.assertFalse(valid_range_check(9, 9, 10, 10, 2), f"Expected False for (9, 9) in 10x10 with ws=2")
        self.assertFalse(valid_range_check(0, 0, 10, 10, 2), f"Expected False for (0, 0) in 10x10 with ws=2")

    def test_minimum_valid_case(self):
        self.assertTrue(valid_range_check(2, 2, 5, 5, 2), f"Expected True for (2, 2) in 5x5 with ws=2")

    def test_maximum_valid_case(self):
        self.assertFalse(valid_range_check(2, 2, 4, 4, 2), f"Expected False for (2, 2) in 4x4 with ws=2 (no valid space)")

if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)

```

```

.....
Ran 4 tests in 0.003s
OK

```

```

In [36]: def trackPoints(pt_x, pt_y, im, ws, iteration):
    ...
    Tracking initial points (pt_x, pt_y) across the image sequence
    Inputs:
    pt_x: initialized keypoint position of x
    pt_y: initialized keypoint position of y

```

```

im: image sequence (N frames)
ws: patch window size
iteration: controls how many iterations to run in iterative_LucasKanade

Outputs:
    track_x: [Number of keypoints] x [2]
    track_y: [Number of keypoints] x [2]
    ...
N = np.prod(pt_x.shape)
nim = len(im)
track_x = np.zeros((N, nim))
track_y = np.zeros((N, nim))
track_x[:,0] = pt_x
track_y[:,0] = pt_y
for t in range(nim - 1):
    track_x[:, t+1], track_y[:, t+1] = getNextPoints(track_x[:, t], track_y[:, t], im[t,:,:), im[t+1,:,:), ws, iteration)
return track_x, track_y

```

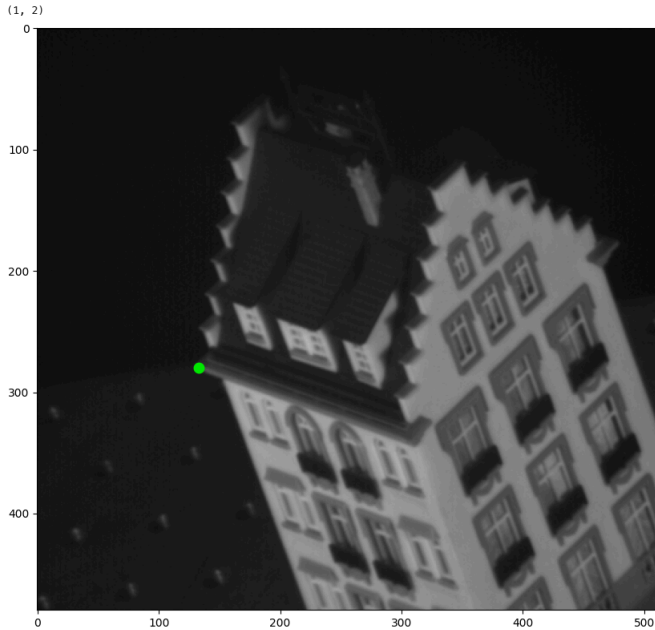
Sanity Check 3: Tracking 1 point with synthetic motion

```

In [37]: # Let's start by using only one point
index = 16
key_pt = key_pts[index:index+1]
print(key_pt.shape)
im0 = im[0]

fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(im0 / 255)
plt.scatter(key_pt[:, 0], key_pt[:, 1], color=(0,0.9,0), linewidths=3, s=50)
plt.show()

```



```

In [38]: # Let's make a synthetic motion, e.g., moving the whole image 1 pixel to the right
motion = np.array([1, 0])
key_pt_new = key_pt + motion

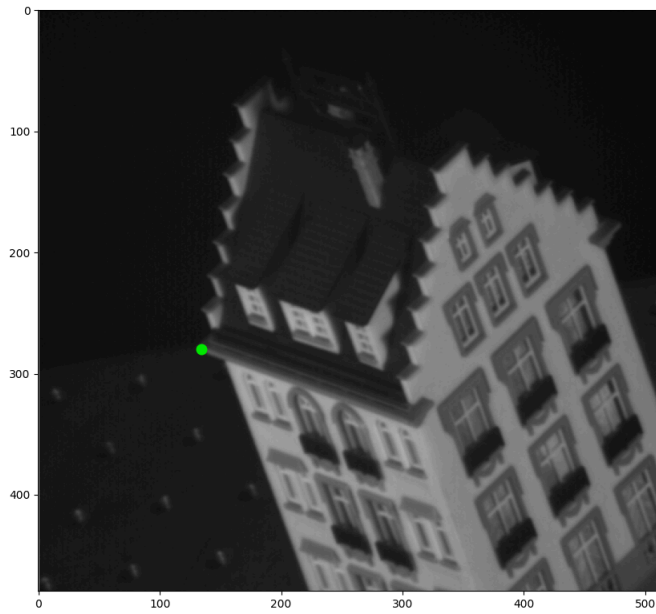
width, height = im0.shape[:2]

# shifted image by one pixel
# Translation matrix (shift 1 pixel to the right)
M = np.float32([[1, 0, motion[0,0]], [0, 1, 0]]) # Moves the image 1 pixel right

# Apply the affine transformation
shifted_image = cv2.warpAffine(im0, M, (height, width), borderValue=(255, 255, 255))

# visualize shifted im0
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(shifted_image / 255)
plt.scatter(key_pt_new[:, 0], key_pt_new[:, 1], color=(0,0.9,0), linewidths=3, s=50)
plt.show()

```



```
In [39]: print('new position: ', key_pt_new)
         print('old position: ', key_pt)
```

```
new position:  [[134 280]]
old position:  [[133 280]]
```

At this moment, we know the solution from the tracking should be `[[134 280]]`.

Let's try to implement our tracking algorithm and use it to find it.

```
In [40]: # Parameters setting
         # You can start with a smaller number for fast debugging
         window_size = 15
         iteration = 10

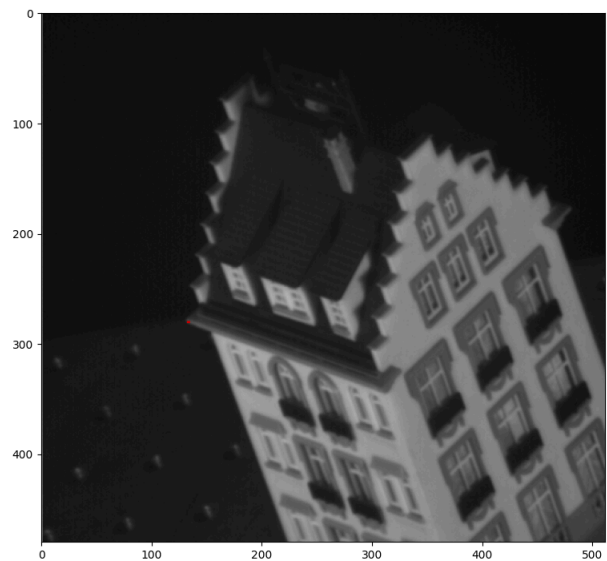
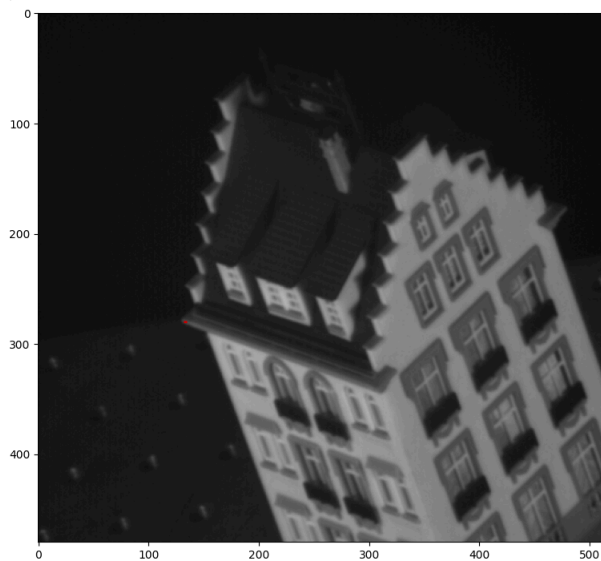
         '''
         PLEASE DO NOT TOUCH CODES BELOW
         '''
         in_img = np.zeros(im.shape).astype(np.float32)
         in_img[0] = im
         in_img[1] = shifted_image
         in_img = in_img[:2]
         tracked_x, tracked_y = trackPoints(pt_x=key_pt[:,0], pt_y=key_pt[:,1], im=in_img, ws=window_size, iteration=iteration)

         print('prediction: ', np.round(np.concatenate((tracked_x[:, 1:], tracked_y[:, 1:]), axis=1)))
         print('ground-truth: ', key_pt_new)

         # Plot your results
         fig, axs = plt.subplots(1, 2, figsize=(20, 10))
         axs[0].imshow(im / 255)
         axs[1].imshow(shifted_image / 255)

         for i in range(tracked_x.shape[1] - 1):
             axs[0].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
             axs[1].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
         plt.show()
```

```
/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
      x2_idx = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
      y2_idx = y_next + uv[idx][1] #complete here
iteration: 0 , shift_uv_mean: 0.9798807918162378
iteration: 1 , shift_uv_mean: 0.021147605624948287
prediction:  [[134. 280.]]
ground-truth:  [[134 280]]
```

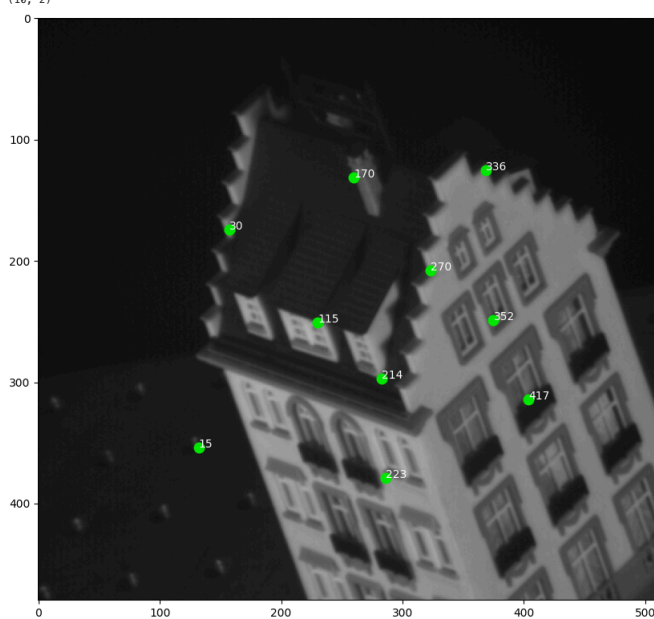


Sanity Check 4: Tracking 10 points with synthetic motion

```
In [41]: # Congrats on solving all the above checking! We're almost there!
# Now let's try with more than one points
index = [15, 30, 170, 115, 214, 223, 270, 352, 336, 417]
key_pt = key_pts[index]
print(key_pt.shape)
im0 = im[0]

fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(im0 / 255)
plt.scatter(key_pt[:, 0], key_pt[:, 1], color=(0,0.9,0), linewidths=3, s=50)
for i in range(key_pt.shape[0]):
    plt.text(key_pt[i,0], key_pt[i,1], str(index[i]), c='white')

plt.show()
```



```
In [42]: # Let's make other synthetic motion, e.g., moving the whole image 5 pixel right, 5 pixel down
motion = np.array([1, 0])
key_pt_new = key_pt + motion

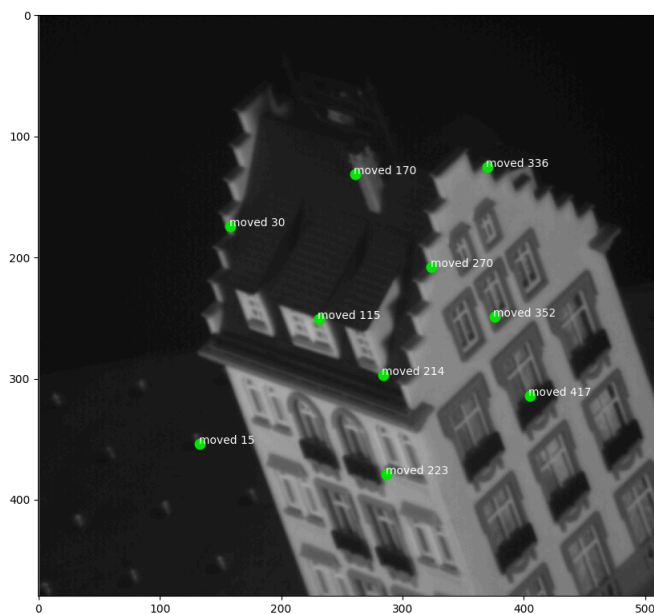
width, height = im0.shape[:2]

# Translation matrix
M = np.float32([[1, 0, motion[0,0]], [0, 1, motion[0,1]]])

# Apply the affine transformation
shifted_image = cv2.warpAffine(im0, M, (height, width), borderValue=(255, 255, 255))

# visualize shifted im0
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(shifted_image / 255)
plt.scatter(key_pt_new[:, 0], key_pt_new[:, 1], color=(0,0.9,0), linewidths=3, s=50)
for i in range(key_pt.shape[0]):
    plt.text(key_pt[i,0], key_pt[i,1], 'moved ' + str(index[i]), c='white')

plt.show()
```



```
In [43]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 100

****
PLEASE DO NOT TOUCH CODES BELOW
****
in_img = np.zeros(im.shape).astype(np.float32)
in_img[0] = im0
in_img[1] = shifted_image
```

```

in_img = in_img[:2]
tracked_x, tracked_y = trackPoints(pt_x=key_pt[:,0], pt_y=key_pt[:,1], im=in_img, ws=window_size, iteration=iteration)

print('prediction: ', np.round(np.concatenate((tracked_x[:, 1:], tracked_y[:, 1:]), axis=1)))
print('ground-truth: ', key_pt_new)

# Plot your results
fig, axs = plt.subplots(1, 2, figsize=(20, 10))
axs[0].imshow(im0 / 255)
axs[1].imshow(shifted_image / 255)

for i in range(tracked_x.shape[1] - 1):
    axs[0].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
    axs[1].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
plt.show()

```

```

/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  x2[idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  y2[idx] = y_next + uv[idx][1] #complete here

```

```

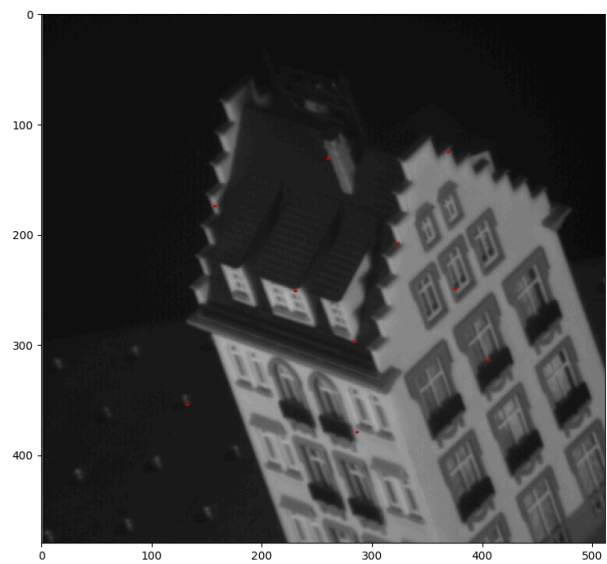
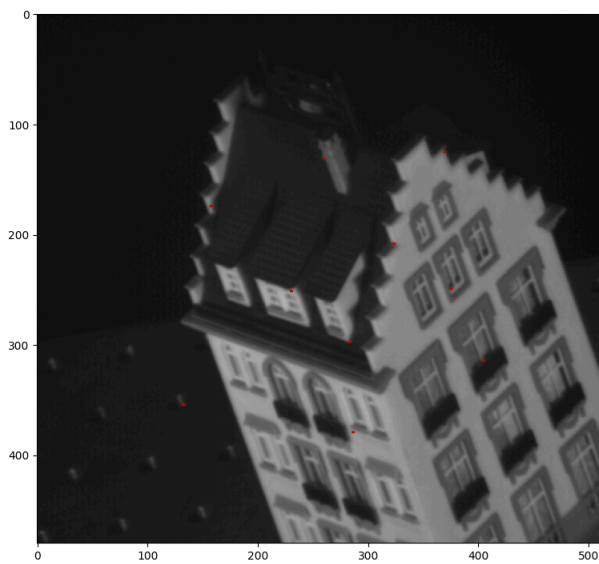
iteration: 0, shift_uv_mean: 0.9999080627244495
iteration: 1, shift_uv_mean: 0.021391686993491338
prediction: [[133. 354.]

```

```

[158. 174.]
[261. 131.]
[231. 251.]
[284. 297.]
[287. 379.]
[324. 208.]
[376. 249.]
[370. 125.]
[405. 314.]]
ground-truth: [[133 354]
[158 174]
[261 131]
[231 251]
[284 297]
[287 379]
[324 208]
[376 249]
[370 125]
[405 314]]

```



Track real data - Track 1 point in two frames

```

In [44]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

'''
PLEASE DO NOT TOUCH CODES BELOW
'''

in_img = np.zeros(im.shape).astype(np.float32)
in_img[0] = im[0]
in_img[1] = im[1]
in_img = in_img[:2]

start_id = 15
end_id = 16
tracked_x, tracked_y = trackPoints(pt_x=key_pts[start_id:end_id,0], pt_y=key_pts[start_id:end_id,1], im=in_img, ws=window_size, iteration=iteration)

# Plot your results
fig, axs = plt.subplots(1, 2, figsize=(20, 10))
axs[0].imshow(im[0] / 255)
axs[1].imshow(im[1] / 255)

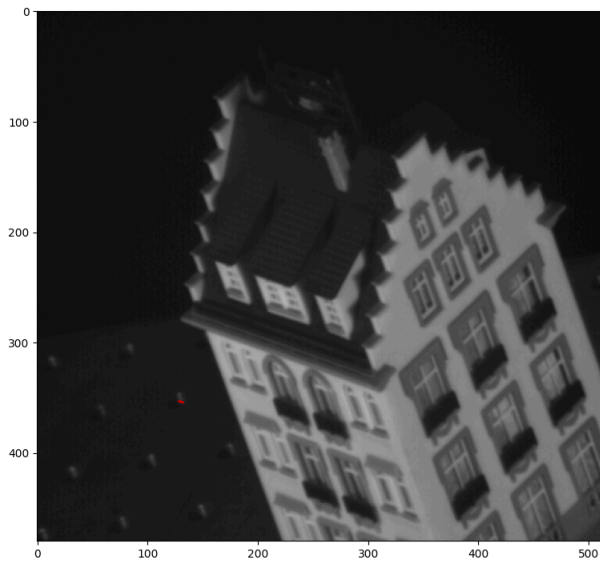
for i in range(tracked_x.shape[1] - 1):
    axs[0].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
    axs[1].plot([tracked_x[i, 1], tracked_x[i, 1 + 1]], [tracked_y[i, 1], tracked_y[i, 1 + 1]], color=(0.8,0,0), linewidth=2)
plt.show()

```

```

/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  x2[idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  y2[idx] = y_next + uv[idx][1] #complete here
iteration: 0, shift_uv_mean: 2.9842176647519683
iteration: 1, shift_uv_mean: 0.582980834581579
iteration: 2, shift_uv_mean: 0.011755563898483711

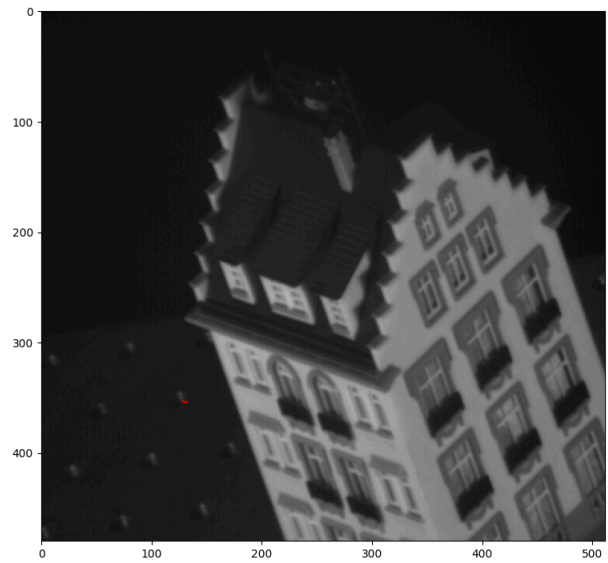
```

Original point: (132, 354)

Motion (u, v): (-2, -1)

Reference image



Track real data - Track 5 points in two frames

```
In [45]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

'''
PLEASE DO NOT TOUCH CODES BELOW
'''

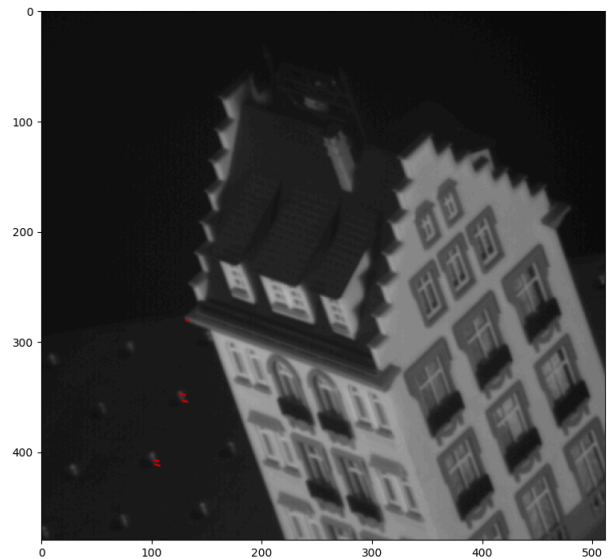
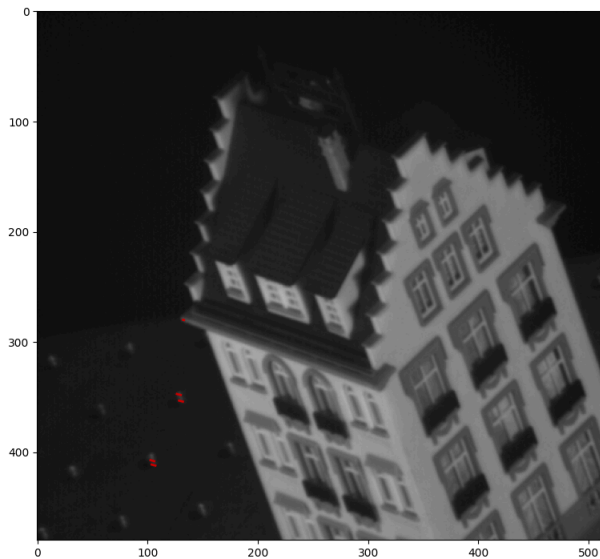
in_img = np.zeros(im.shape).astype(np.float32)
in_img[0] = im[0]
in_img[1] = im[1]
in_img = in_img[:2]

start_id = 12
end_id = 17
tracked_x, tracked_y = trackPoints(pt_x=key_pts[start_id:end_id,0], pt_y=key_pts[start_id:end_id,1], im=in_img, ws=window_size, iteration=iteration)

# Plot your results
fig, axes = plt.subplots(1, 2, figsize=(20, 10))
axes[0].imshow(im[0] / 255)
axes[1].imshow(im[1] / 255)

for i in range(tracked_x.shape[1] - 1):
    axes[0].plot([tracked_x[i], 1], tracked_x[i+1], tracked_y[i], tracked_y[i+1]], color=(0.8,0,0), linewidth=2)
    axes[1].plot([tracked_x[i], 1], tracked_x[i+1], tracked_y[i], tracked_y[i+1]], color=(0.8,0,0), linewidth=2)
plt.show()
```

```
/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before perform
ing this operation. (Deprecated NumPy 1.25.)
    x2[idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before perform
ing this operation. (Deprecated NumPy 1.25.)
    y2[idx] = y_next + uv[idx][1] #complete here
iteration: 0 , shift_uv_mean: 2.52752908880423
iteration: 1 , shift_uv_mean: 0.4712826247702064
iteration: 2 , shift_uv_mean: 0.009101740641176349
```



original point: (106, 408), (107, 412), (130, 348), (132, 354), (133, 280)

motion(u, v): (-3, -1), (-3, -1), (-3, -1), (-3, -1), (-3, 0)

reference image:

Track real data - Track all the points in two frames

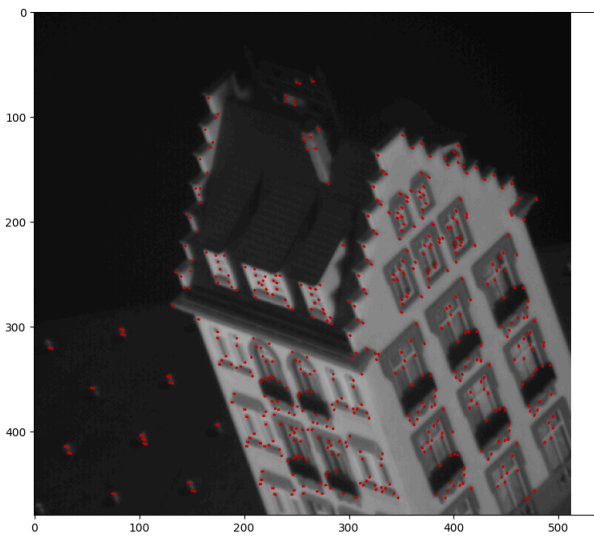
```
In [46]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

'''
PLEASE DO NOT TOUCH CODES BELOW
'''
in_img = np.zeros(im.shape).astype(np.float32)
in_img[0] = im[0]
in_img[1] = im[1]
in_img = in_img[:2]
tracked_x, tracked_y = trackPoints(pt_x=key_pts[:, 0], pt_y=key_pts[:, 1], im=in_img, ws=window_size, iteration=iteration)

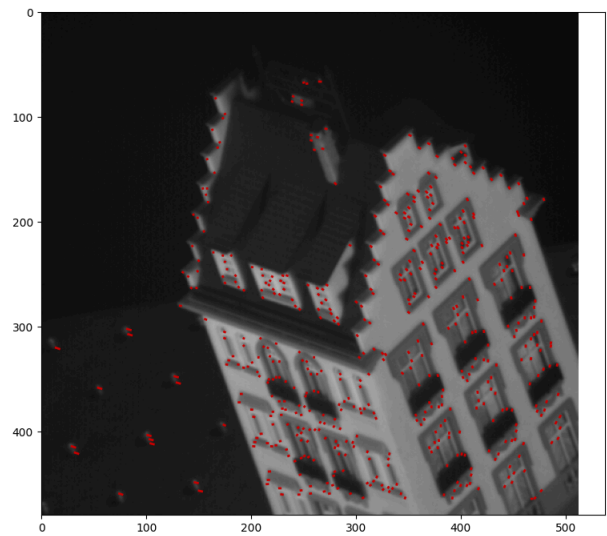
# Plot your results
fig, axs = plt.subplots(1, 2, figsize=(20, 10))
axs[0].imshow(im[0] / 255)
axs[1].imshow(im[1] / 255)

for i in range(tracked_x.shape[1] - 1):
    axs[0].plot([tracked_x[i, 1], tracked_x[i, i + 1]], [tracked_y[i, 1], tracked_y[i, i + 1]], color=(0.8, 0.0, 0), linewidth=2)
    axs[1].plot([tracked_x[i, 1], tracked_x[i, i + 1]], [tracked_y[i, 1], tracked_y[i, i + 1]], color=(0.8, 0.0, 0), linewidth=2)
plt.show()

/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  x2[idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  y2[idx] = y_next + uv[idx][1] #complete here
iteration: 0 , shift_uv_mean: 0.7334187785032527
iteration: 1 , shift_uv_mean: 0.027564091478959066
```



Reference image



Write-up (10 pts)

- (10 pts) For all the keypoints, display (1) the keypoints at the first frame (as green) and (2) the tracked keypoints at the second frame (as red) on the first frame of the sequence.

```
In [47]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

in_img = np.zeros(im.shape).astype(np.float32)
in_img[0] = im[0]
in_img[1] = im[1]
in_img = in_img[:2]
tracked_x, tracked_y = trackPoints(pt_x=key_pts[:, 0], pt_y=key_pts[:, 1], im=in_img, ws=window_size, iteration=iteration)

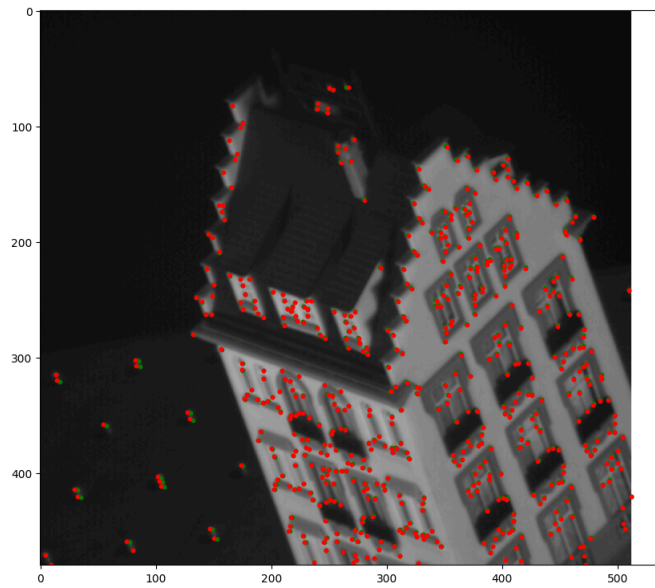
# Plot the results
fig, ax = plt.subplots(figsize=(10, 10))

# Display the first frame
ax.imshow(im[0] / 255)

ax.scatter(tracked_x[:, 0], tracked_y[:, 0], color='green', s=10)
ax.scatter(tracked_x[:, 1], tracked_y[:, 1], color='red', s=10)

plt.show()

/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  x2[idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  y2[idx] = y_next + uv[idx][1] #complete here
iteration: 0 , shift_uv_mean: 0.7334187785032527
iteration: 1 , shift_uv_mean: 0.027564091478959066
```



This is formatted as code

[Bonus] Track real data - Track all points in all frames (10 points)

```
In [48]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

'''
PLEASE DO NOT TOUCH CODES BELOW
'''

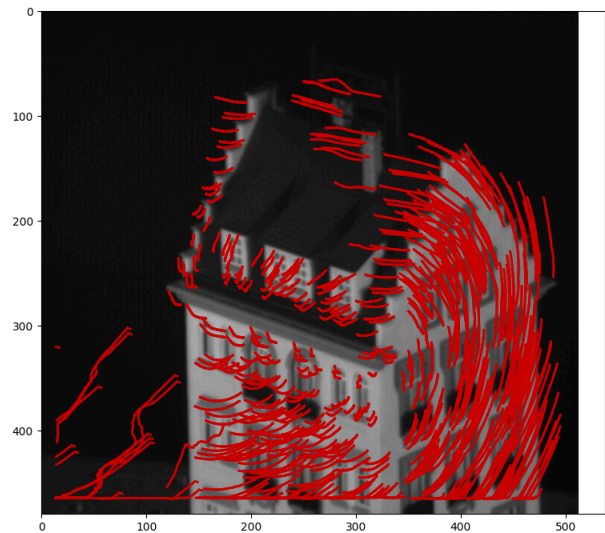
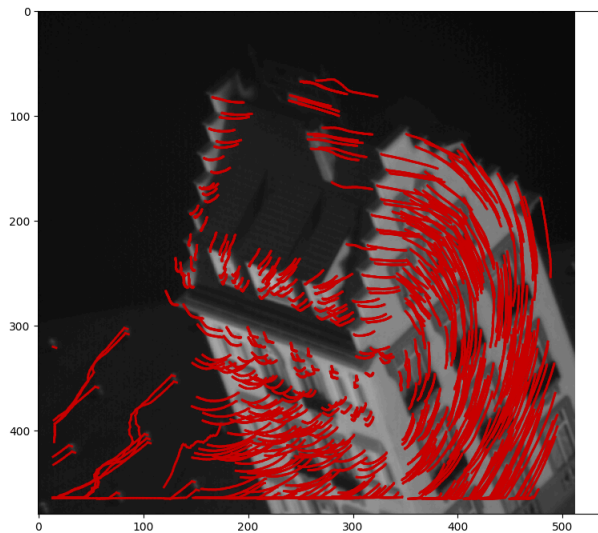
tracked_x, tracked_y = trackPoints(pt_x=key_pts[:,0], pt_y=key_pts[:,1], im=im, ws=window_size, iteration=iteration)

# Plot your results
fig, axs = plt.subplots(1, 2, figsize=(20, 10))
axs[0].imshow(im[0] / 255)
axs[1].imshow(im[-1] / 255)

for i in range(tracked_x.shape[1] - 1):
    axs[0].plot([tracked_x[:, i], tracked_x[:, i + 1]], [tracked_y[:, i], tracked_y[:, i + 1]], color=(0.8,0,0), linewidth=2)
    axs[1].plot([tracked_x[:, i], tracked_x[:, i + 1]], [tracked_y[:, i], tracked_y[:, i + 1]], color=(0.8,0,0), linewidth=2)
plt.show()

/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    x2[idx] = x_next + uv[idx][0]      #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    y2[idx] = y_next + uv[idx][1]      #complete here
```

```
iteration: 0 , shift_uv_mean: 0.7334187785032527
iteration: 1 , shift_uv_mean: 0.027564091478959066
iteration: 0 , shift_uv_mean: 1.0554099189887785
iteration: 1 , shift_uv_mean: 0.0657014660917394
iteration: 0 , shift_uv_mean: 1.0196099332759399
iteration: 1 , shift_uv_mean: 0.06011625190525963
iteration: 0 , shift_uv_mean: 1.0921646240735063
iteration: 1 , shift_uv_mean: 0.06941261598068093
iteration: 0 , shift_uv_mean: 0.9912531734249213
iteration: 1 , shift_uv_mean: 0.05992718358623231
iteration: 0 , shift_uv_mean: 0.991376178547213
iteration: 1 , shift_uv_mean: 0.06103785658216647
iteration: 0 , shift_uv_mean: 0.9633991628458954
iteration: 1 , shift_uv_mean: 0.05631815330329303
iteration: 0 , shift_uv_mean: 0.9822926705816919
iteration: 1 , shift_uv_mean: 0.05528930748501044
iteration: 0 , shift_uv_mean: 0.9759466185440595
iteration: 1 , shift_uv_mean: 0.05748772521743205
iteration: 0 , shift_uv_mean: 0.9419684030410378
iteration: 1 , shift_uv_mean: 0.05411020940949794
iteration: 0 , shift_uv_mean: 0.9443157365065342
iteration: 1 , shift_uv_mean: 0.05354884266392692
iteration: 0 , shift_uv_mean: 0.9152576982130431
iteration: 1 , shift_uv_mean: 0.053009347458437054
iteration: 0 , shift_uv_mean: 0.9648729470609911
iteration: 1 , shift_uv_mean: 0.05657426752106022
iteration: 0 , shift_uv_mean: 0.9046206226224562
iteration: 1 , shift_uv_mean: 0.051224743353291254
iteration: 0 , shift_uv_mean: 0.9135404694003102
iteration: 1 , shift_uv_mean: 0.054040090281655937
iteration: 0 , shift_uv_mean: 0.9244460926770754
iteration: 1 , shift_uv_mean: 0.05313022335806473
iteration: 0 , shift_uv_mean: 0.928148162143616
iteration: 1 , shift_uv_mean: 0.05317589947685622
iteration: 0 , shift_uv_mean: 0.9000905644980845
iteration: 1 , shift_uv_mean: 0.04729653405844313
iteration: 0 , shift_uv_mean: 0.9026561562717479
iteration: 1 , shift_uv_mean: 0.04804606518054432
iteration: 0 , shift_uv_mean: 0.876947100930215
iteration: 1 , shift_uv_mean: 0.043078980721727424
iteration: 0 , shift_uv_mean: 0.9425819352826282
iteration: 1 , shift_uv_mean: 0.0465211362869328
iteration: 0 , shift_uv_mean: 0.8992810209732981
iteration: 1 , shift_uv_mean: 0.83802727280936872
iteration: 0 , shift_uv_mean: 0.863297324900067
iteration: 1 , shift_uv_mean: 0.83351607012538144
iteration: 0 , shift_uv_mean: 0.8653085950678676
iteration: 1 , shift_uv_mean: 0.83627087512212603
iteration: 0 , shift_uv_mean: 0.86755636380804974
iteration: 1 , shift_uv_mean: 0.8355820832695288
iteration: 0 , shift_uv_mean: 0.885677541231259
iteration: 1 , shift_uv_mean: 0.83979394184391073
iteration: 0 , shift_uv_mean: 0.8564466262625835
iteration: 1 , shift_uv_mean: 0.836547206727189356
iteration: 0 , shift_uv_mean: 0.8503285276832175
iteration: 1 , shift_uv_mean: 0.8334129129584826
iteration: 0 , shift_uv_mean: 0.8601641021618831
iteration: 1 , shift_uv_mean: 0.83475947416252734
iteration: 0 , shift_uv_mean: 0.8445103297776508
iteration: 1 , shift_uv_mean: 0.826728616697883005
iteration: 0 , shift_uv_mean: 0.8432810214615759
iteration: 1 , shift_uv_mean: 0.829822979516192838
iteration: 0 , shift_uv_mean: 0.8380732233184928
iteration: 1 , shift_uv_mean: 0.83474875916831233
iteration: 0 , shift_uv_mean: 0.8227988441952568
iteration: 1 , shift_uv_mean: 0.82854259821127247
iteration: 0 , shift_uv_mean: 0.8057289267182132
iteration: 1 , shift_uv_mean: 0.83137208000604766
iteration: 0 , shift_uv_mean: 0.8043207559926454
iteration: 1 , shift_uv_mean: 0.830190654705200742
iteration: 0 , shift_uv_mean: 0.8292535041282053
iteration: 1 , shift_uv_mean: 0.830229103278760043
iteration: 0 , shift_uv_mean: 0.7882966402283744
iteration: 1 , shift_uv_mean: 0.826701232231308297
iteration: 0 , shift_uv_mean: 0.8195912328325593
iteration: 1 , shift_uv_mean: 0.82945875263256564
iteration: 0 , shift_uv_mean: 0.7999718647828278
iteration: 1 , shift_uv_mean: 0.824618325840282403
iteration: 0 , shift_uv_mean: 0.7803447250258129
iteration: 1 , shift_uv_mean: 0.819408128385205444
iteration: 0 , shift_uv_mean: 0.7793349034050027
iteration: 1 , shift_uv_mean: 0.825364945584105174
iteration: 0 , shift_uv_mean: 0.7847824589031309
iteration: 1 , shift_uv_mean: 0.823773177716253823
iteration: 0 , shift_uv_mean: 0.7805128090908451
iteration: 1 , shift_uv_mean: 0.821770798690983632
iteration: 0 , shift_uv_mean: 0.7948310604957498
iteration: 1 , shift_uv_mean: 0.820003636202236025
iteration: 0 , shift_uv_mean: 0.7779814647329949
iteration: 1 , shift_uv_mean: 0.823154150814842074
iteration: 0 , shift_uv_mean: 0.7753188966431166
iteration: 1 , shift_uv_mean: 0.821271540959791158
iteration: 0 , shift_uv_mean: 0.7678943872770316
iteration: 1 , shift_uv_mean: 0.81752857389741771
iteration: 0 , shift_uv_mean: 0.7781428202185345
iteration: 1 , shift_uv_mean: 0.82030641040119309
iteration: 0 , shift_uv_mean: 0.7580075439808758
iteration: 1 , shift_uv_mean: 0.818992106484478962
iteration: 0 , shift_uv_mean: 0.7760050417678992
iteration: 1 , shift_uv_mean: 0.821612460672254998
```



Reference image

[Bonus] Write-up (20 pts)

- (10 pts) For 20 random keypoints, draw the 2D path over the sequence of frames. That is, plot the progression of image coordinates for each of the 20 keypoints. Plot each of the paths on the same figure, overlaid on the first frame of the sequence.

```
In [49]: # Parameters setting
# You can start with a smaller number for fast debugging
window_size = 15
iteration = 10

# Get 20 random keypoints
random_indices = np.random.choice(key_pts.shape[0], size=20, replace=False)

tracked_x, tracked_y = trackPoints(pt=key_pts[random_indices, 0], pt_y=key_pts[random_indices, 1], im=im, ws=window_size, iteration=iteration)

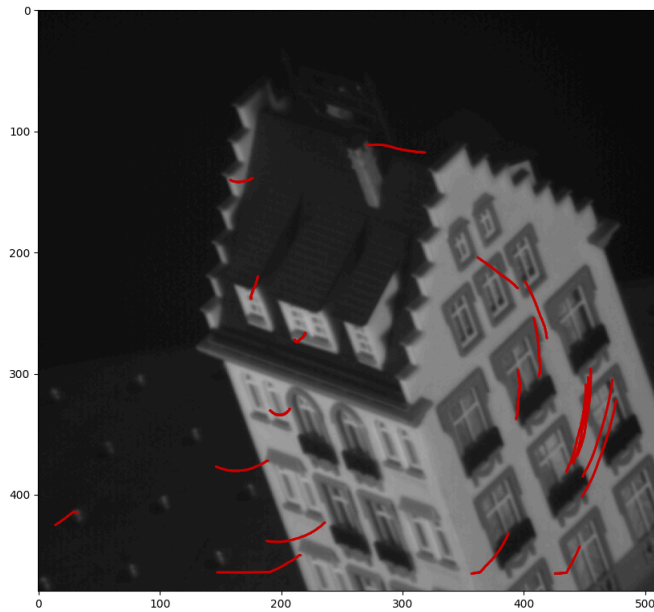
# Plot the results
fig, ax = plt.subplots(figsize=(10, 10))

# Display the first frame
ax.imshow(im[0] / 255)

# Plot the lines
for i in range(tracked_x.shape[1] - 1):
    ax.plot([tracked_x[:, i], tracked_x[:, i + 1]], [tracked_y[:, i], tracked_y[:, i + 1]], color=(0.8, 0, 0), linewidth=2)

plt.show()
```

```
/tmp/ipykernel_322/2090711246.py:211: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before perform
ing this operation. (Deprecated NumPy 1.25.)
  x2_idx] = x_next + uv[idx][0] #complete here
/tmp/ipykernel_322/2090711246.py:212: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before perform
ing this operation. (Deprecated NumPy 1.25.)
  y2_idx] = y_next + uv[idx][1] #complete here
iteration: 0, shift_uv_mean: 0.8049851795935361
iteration: 1, shift_uv_mean: 0.040456952828556495
iteration: 0, shift_uv_mean: 1.2896354267943448
iteration: 1, shift_uv_mean: 0.09355432006554011
iteration: 0, shift_uv_mean: 1.242941586878
iteration: 1, shift_uv_mean: 0.8038837123443098
iteration: 0, shift_uv_mean: 1.3441542905891772
iteration: 1, shift_uv_mean: 0.10045323833770817
iteration: 0, shift_uv_mean: 1.2417186163910379
iteration: 1, shift_uv_mean: 0.0897521983514518
iteration: 0, shift_uv_mean: 1.237668070562472
iteration: 1, shift_uv_mean: 0.08853458647279583
iteration: 0, shift_uv_mean: 1.2043231925014644
iteration: 1, shift_uv_mean: 0.03199504268142786
iteration: 0, shift_uv_mean: 1.1274364623122144
iteration: 1, shift_uv_mean: 0.0334543635439512
iteration: 0, shift_uv_mean: 1.152555679781849
iteration: 1, shift_uv_mean: 0.03783314487059292
iteration: 0, shift_uv_mean: 1.1140173558486033
iteration: 1, shift_uv_mean: 0.0323547258071863
iteration: 0, shift_uv_mean: 1.098466125813825
iteration: 1, shift_uv_mean: 0.02980524478442481
iteration: 0, shift_uv_mean: 1.0935506641007973
iteration: 1, shift_uv_mean: 0.033278351159810995
iteration: 0, shift_uv_mean: 1.1444580225681528
iteration: 1, shift_uv_mean: 0.033294531304210805
iteration: 0, shift_uv_mean: 1.0855763679412218
iteration: 1, shift_uv_mean: 0.03364618310533961
iteration: 0, shift_uv_mean: 1.0520684486283695
iteration: 1, shift_uv_mean: 0.031867087446574606
iteration: 0, shift_uv_mean: 1.018122219251367
iteration: 1, shift_uv_mean: 0.03003862278810935
iteration: 0, shift_uv_mean: 1.0749281315534178
iteration: 1, shift_uv_mean: 0.03508455566946147
iteration: 0, shift_uv_mean: 1.002963022122096
iteration: 1, shift_uv_mean: 0.028613738479705817
iteration: 0, shift_uv_mean: 1.028617273573055
iteration: 1, shift_uv_mean: 0.031147154635279948
iteration: 0, shift_uv_mean: 0.9947435483552169
iteration: 1, shift_uv_mean: 0.027239104689138396
iteration: 0, shift_uv_mean: 1.0077152867899235
iteration: 1, shift_uv_mean: 0.022951630871478088
iteration: 0, shift_uv_mean: 1.0118419470416495
iteration: 1, shift_uv_mean: 0.02308039787777912
iteration: 0, shift_uv_mean: 1.0074558822363529
iteration: 1, shift_uv_mean: 0.024429610744812254
iteration: 0, shift_uv_mean: 0.9953996705441952
iteration: 1, shift_uv_mean: 0.02127926990966428
iteration: 0, shift_uv_mean: 0.9483712385053475
iteration: 1, shift_uv_mean: 0.023653299681356987
iteration: 0, shift_uv_mean: 0.9999112145051603
iteration: 1, shift_uv_mean: 0.021851368082661927
iteration: 0, shift_uv_mean: 0.9699731055584945
iteration: 1, shift_uv_mean: 0.0199464132015991
iteration: 0, shift_uv_mean: 0.9938214991632746
iteration: 1, shift_uv_mean: 0.02245635764377991
iteration: 0, shift_uv_mean: 0.979390972183773
iteration: 1, shift_uv_mean: 0.02356191464760414
iteration: 0, shift_uv_mean: 0.9898151825384924
iteration: 1, shift_uv_mean: 0.021356563981925616
iteration: 0, shift_uv_mean: 0.9779190943978916
iteration: 1, shift_uv_mean: 0.019553253897210115
iteration: 0, shift_uv_mean: 0.9691584300805515
iteration: 1, shift_uv_mean: 0.020475733151561953
iteration: 0, shift_uv_mean: 0.965983464007804
iteration: 1, shift_uv_mean: 0.022378081331942037
iteration: 0, shift_uv_mean: 0.970035007547644
iteration: 1, shift_uv_mean: 0.019821632147442415
iteration: 0, shift_uv_mean: 0.9465345049197684
iteration: 1, shift_uv_mean: 0.02692954465233452
iteration: 0, shift_uv_mean: 0.940187367268755
iteration: 1, shift_uv_mean: 0.022069326020423847
iteration: 0, shift_uv_mean: 0.9520758352159222
iteration: 1, shift_uv_mean: 0.018260784723669272
iteration: 0, shift_uv_mean: 0.9382684547886497
iteration: 1, shift_uv_mean: 0.01934512231802117
iteration: 0, shift_uv_mean: 0.9436724203372715
iteration: 1, shift_uv_mean: 0.018646617015191647
iteration: 0, shift_uv_mean: 0.9309495017101602
iteration: 1, shift_uv_mean: 0.019145714341519748
iteration: 0, shift_uv_mean: 0.928808027322256
iteration: 1, shift_uv_mean: 0.02141823099898921
iteration: 0, shift_uv_mean: 0.9394421069646659
iteration: 1, shift_uv_mean: 0.019099805121121795
iteration: 0, shift_uv_mean: 0.9455738622367676
iteration: 1, shift_uv_mean: 0.016280357798053622
iteration: 0, shift_uv_mean: 0.9424741034984733
iteration: 1, shift_uv_mean: 0.01728992412791
iteration: 0, shift_uv_mean: 0.9301881838735755
iteration: 1, shift_uv_mean: 0.01365508267548913
iteration: 0, shift_uv_mean: 0.9529467542089974
iteration: 1, shift_uv_mean: 0.016340167165479036
iteration: 0, shift_uv_mean: 0.9458551247176449
iteration: 1, shift_uv_mean: 0.01815265422144563
```



Export Notebook

```
In [50]: !sudo apt-get update && apt-get install -y texlive texlive-xetex texlive-latex-extra texlive-fonts-recommended texlive-plain-generic pandoc
!pip install py pandoc
0% [Working]

Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
0% [Waiting for headers] [Connecting to security.ubuntu.com (185.125.190.83)] [
Hit:2 https://cli.github.com/packages stable InRelease
0% [Waiting for headers] [Connecting to security.ubuntu.com (185.125.190.83)] [
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
0% [Connecting to security.ubuntu.com (185.125.190.83)] [Connected to cloud.r-p

Hit:4 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Hit:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:7 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-fonts-recommended is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-plain-generic is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 119 not upgraded.
Requirement already satisfied: py pandoc in /usr/local/lib/python3.12/dist-packages (1.16.2)

In [51]: from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

In [52]: !cp /content/drive/MyDrive/CMSC426/CMSC426_Assignment2_sp26.ipynb ./
!jupyter nbconvert --to PDF "CMSC426_Assignment2_sp26.ipynb"

[NbConvertApp] Converting notebook CMSC426_Assignment2_sp26.ipynb to PDF
[NbConvertApp] Support files will be in CMSC426_Assignment2_sp26_files/
[NbConvertApp] Making directory ./CMSC426_Assignment2_sp26_files
[NbConvertApp] Writing 135656 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 4555026 bytes to CMSC426_Assignment2_sp26.pdf
```